Name : Shweta Satyaprakash Gawade

Rollno : Cs18 Batch:01

Aim : To implement a **Logistic Regression** model to classify credit card transactions as **fraudulent or legitimate**.

Objectives

1. To understand the mathematical foundations of **Logistic Regression** and the **Sigmoid Function**.
2. To understand and handle **class imbalance** in credit card fraud datasets.
3. To implement **Gradient Descent** for optimizing Logistic Regression parameters.
4. To evaluate the model using **Confusion Matrix, Precision, Recall, F1-Score, ROC Curve, and AUC**.
5. To understand the importance of **Recall and False Negative Rate** in fraud detection.

Relevant Theory

Logistic Regression

Logistic Regression is a supervised machine learning algorithm used for **binary classification**. In this experiment, it is used to classify credit card transactions into two classes:

- **0** $\rightarrow$ **Legitimate transaction**
- **1** $\rightarrow$ **Fraudulent transaction**

The model first calculates a linear combination of input features:

$$z = w^T X + b$$

where:

- w = weight vector
- X = input feature vector
- b = bias/intercept
- z = linear predictor

Sigmoid Function

The Sigmoid Function converts the value of z into a probability between 0 and 1:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The predicted probability that a transaction is fraudulent is:

$$P(y = 1|X) = \sigma(w^T X + b)$$

The predicted class can be obtained using a threshold:

$$\hat{y} = \begin{cases} 1, & \text{if } P(y = 1|X) \geq 0.5 \\ 0, & \text{if } P(y = 1|X) < 0.5 \end{cases}$$

Cost Function

Logistic Regression uses the **Binary Cross-Entropy Loss** or **Log Loss**.

The cost function is:

$$J(w, b) = -\frac{1}{m} \sum_{i=1}^{m} [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

where:

- m = number of training examples
- y_i = actual class label
- $\hat{y}_i$ = predicted probability

The objective of training is to find values of w and b that minimize the cost function.

Gradient Descent

Gradient Descent is used to optimize the parameters of the Logistic Regression model.

The weights are updated using:

$$w := w - \alpha \frac{\partial J}{\partial w}$$

The bias is updated using:

$$b := b - \alpha \frac{\partial J}{\partial b}$$

where α is the learning rate.

For Logistic Regression, the gradient of the cost function with respect to the weights can be expressed as:

$$\frac{\partial J}{\partial w} = \frac{1}{m} X^T (\hat{y} - y)$$

The gradient with respect to the bias is:

$$\frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)$$

Therefore, the parameter updates become:

$$w := w - \alpha \left[\frac{1}{m}X^T(\hat{y} - y)\right]$$

$$b := b - \alpha \left[\frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)\right]$$

Class Imbalance

Credit card fraud datasets are generally highly imbalanced because legitimate transactions significantly outnumber fraudulent transactions.

For example, the dataset may contain a very large number of legitimate transactions and only a small number of fraudulent transactions.

A model that predicts most transactions as legitimate may achieve high accuracy but perform poorly at detecting fraud.

Therefore, techniques such as:

- Random Undersampling
- Oversampling
- Class Weighting

can be used to handle class imbalance.

Evaluation Metrics

Confusion Matrix

The confusion matrix consists of four components:

- **True Positive (TP):** Fraudulent transaction correctly identified as fraud.
- **True Negative (TN):** Legitimate transaction correctly identified as legitimate.
- **False Positive (FP):** Legitimate transaction incorrectly identified as fraud.
- **False Negative (FN):** Fraudulent transaction incorrectly identified as legitimate.

Accuracy

Accuracy measures the proportion of correctly classified transactions:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

However, accuracy alone is not sufficient for highly imbalanced fraud detection datasets.

Precision

Precision indicates the proportion of transactions predicted as fraudulent that are actually fraudulent:

$$Precision = \frac{TP}{TP + FP}$$

A high Precision means that the model produces fewer false fraud alerts.

Recall

Recall measures the proportion of actual fraudulent transactions correctly detected:

$$Recall = \frac{TP}{TP + FN}$$

Recall is particularly important in fraud detection because a **False Negative** represents a fraudulent transaction that was not detected.

F1-Score

F1-Score is the harmonic mean of Precision and Recall:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

ROC Curve

The Receiver Operating Characteristic (ROC) curve represents the relationship between the **True Positive Rate (TPR)** and **False Positive Rate (FPR)** at different classification thresholds.

The True Positive Rate is:

$$TPR = \frac{TP}{TP + FN}$$

The False Positive Rate is:

$$FPR = \frac{FP}{FP + TN}$$

The ROC curve helps determine how well the model separates fraudulent transactions from legitimate transactions.

Area Under the Curve (AUC)

The **Area Under the ROC Curve (AUC)** summarizes the performance of the classifier across different classification thresholds.

An AUC value close to:

$$AUC = 1$$

indicates excellent discrimination, while:

$$AUC = 0.5$$

indicates performance approximately equivalent to random classification.

Dataset Information

The Credit Card Fraud Dataset contains transaction-related features such as transaction amount, distance from home, purchase-price-related ratios, and transaction characteristics.

The target variable is fraud:

0 ⟶ Legitimate transaction

1 ⟶ Fraudulent transaction

Task 1: Environment Setup and Dataset Acquisition

Use dataset for this experiment Credit Card Fraud Dataset

Import the required Python libraries such as **NumPy, Pandas, Matplotlib, and Scikit-learn**.

Load the **Credit Card Fraud Dataset** into a Pandas DataFrame.

Display the following information:

- First five records of the dataset.
- Shape of the dataset.
- Column names.
- Data types of all columns.
- Descriptive statistics of numerical features.

Identify the **feature variables** and the **target variable** `fraud`.

The target variable is defined as:

$$\text{Class Percentage} = \frac{\text{Number of Transactions in Class}}{\text{Total Number of Transactions}} \times 100$$

fraud =

- 0 -> Legitimate transaction

- 1 -> Fraudulent transaction

The objective of this task is to understand the structure of the dataset and identify the input features and target variable required for building the Logistic Regression model.

Identify the feature variables and the target variable fraud.

```
# ============================================
# Task 1: Environment Setup and Dataset Acquisition
# Logistic Regression - Credit Card Fraud
# ============================================

# Step 1: Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    confusion_matrix,
    classification_report,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_curve,
    roc_auc_score
)

# ============================================
# Step 2: Load the dataset
# ============================================

# Upload the CSV file in Google Colab first
from google.colab import files

uploaded = files.upload()

# Read the uploaded CSV file
file_name = list(uploaded.keys())[0]
df = pd.read_csv(file_name)

print("Dataset loaded successfully!")

# ============================================
# Step 3: Display first five records
# ============================================

print("\nFirst five records:")
display(df.head())
```

```
# ============================================
# Step 4: Display shape of dataset
# ============================================

print("\nShape of dataset:")
print(df.shape)

print("Number of rows:", df.shape[0])
print("Number of columns:", df.shape[1])

# ============================================
# Step 5: Display column names
# ============================================

print("\nColumn names:")
print(df.columns.tolist())

# ============================================
# Step 6: Display data types
# ============================================

print("\nData types of all columns:")
print(df.dtypes)

# ============================================
# Step 7: Descriptive statistics
# ============================================

print("\nDescriptive statistics of numerical features:")
display(df.describe())

# ============================================
# Step 8: Check missing values
# ============================================

print("\nMissing values in each column:")
print(df.isnull().sum())

# ============================================
# Step 9: Identify target variable
# ============================================

target = "fraud"

if target not in df.columns:
    print("\nERROR: 'fraud' column was not found.")
    print("Available columns are:")
    print(df.columns.tolist())
else:
    print("\nTarget variable:", target)

    # Feature variables
    X = df.drop(columns=[target])

    # Target variable
```

```
    y = df[target]

    print("\nFeature variables:")
    print(X.columns.tolist())

    print("\nTarget variable:")
    print(y.name)

# ============================================
# Step 10: Check target classes
# ============================================

print("\nClass distribution:")
print(y.value_counts())

# ============================================
# Step 11: Calculate class percentages
# ============================================

class_counts = y.value_counts()
class_percentages = y.value_counts(normalize=True) * 100

print("\nClass percentages:")

for class_value in class_counts.index:
    if class_value == 0:
        label = "Legitimate"
    elif class_value == 1:
        label = "Fraudulent"
    else:
        label = "Unknown"

    print(
        f"Class {class_value} ({label}): "
        f"{class_counts[class_value]} transactions "
        f"({class_percentages[class_value]:.2f}%)"
    )

# ============================================
# Step 12: Display class distribution graph
# ============================================

plt.figure(figsize=(7, 5))

y.value_counts().sort_index().plot(kind="bar")

plt.xlabel("Class")
plt.ylabel("Number of Transactions")
plt.title("Credit Card Transaction Class Distribution")
plt.xticks(
    ticks=[0, 1],
    labels=["Legitimate (0)", "Fraudulent (1)"],
    rotation=0
)
```

```
plt.show()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving card_transdata.csv to card_transdata.csv
Dataset loaded successfully!

First five records:
```

	distance_from_home	distance_from_last_transaction	ratio_to_median_purcha
0	57.877857	0.311140	
1	10.829943	0.175592	
2	5.091079	0.805153	
3	2.247564	5.600044	
4	44.190936	0.566486	

```
Shape of dataset:
(1000000, 8)
Number of rows: 1000000
Number of columns: 8

Column names:
['distance_from_home', 'distance_from_last_transaction', 'ratio_to_median_pur

Data types of all columns:
distance_from_home              float64
distance_from_last_transaction  float64
ratio_to_median_purchase_price  float64
repeat_retailer                 float64
used_chip                       float64
used_pin_number                 float64
online_order                    float64
fraud                           float64
dtype: object

Descriptive statistics of numerical features:
```

	distance_from_home	distance_from_last_transaction	ratio_to_median_pu
count	1000000.000000	1000000.000000	1
mean	26.628792	5.036519	
std	65.390784	25.843093	
min	0.004874	0.000118	
25%	3.878008	0.296671	
50%	9.967760	0.998650	
75%	25.743985	3.355748	
max	10632.723672	11851.104565	

```
Missing values in each column:
distance_from_home              0
distance_from_last_transaction  0
ratio_to_median_purchase_price  0
repeat_retailer                 0
```

Task 2: Perform Exploratory Data Analysis

Perform **Exploratory Data Analysis (EDA)** to understand the structure, distribution, and characteristics of the Credit Card Fraud Dataset.

Calculate and display the following:

repeat_retailer 0
used_chip 0
used_pin_number 0
online_order 0
fraud 0
dtype: int64
Target variable: fraud
Feature variables:
['distance_from_home', 'distance_from_last_transaction', 'ratio_to_median_pur
Target variable:
fraud
Class distribution:
fraud
0.0 912597
1.0 87403
Name: count, dtype: int64
Class percentages:
Class 0.0 (Legitimate): 912597 transactions (91.26%)
Class 1.0 (Fraudulent): 87403 transactions (8.74%)

- Number of **legitimate transactions**.
- Number of **fraudulent transactions**.
- Percentage of legitimate transactions.
- Percentage of fraudulent transactions.
- Distribution of numerical features.

The percentage of each class can be calculated using:

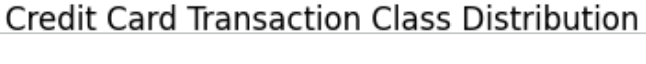
$$\text{Class Percentage} = \frac{\text{Number of Transactions in Class}}{\text{Total Number of Transactions}} \times 100$$

Plot the **class distribution** using a suitable visualization such as a bar chart.

Analyze the class distribution and explain the problem of **classification imbalance** in credit card fraud detection.

A dataset is considered highly imbalanced when the number of legitimate transactions is significantly greater than the number of fraudulent transactions.

Therefore, accuracy alone may not be sufficient to evaluate the performance of the fraud detection model.




```
# ==========================================
# Task 2: Exploratory Data Analysis (EDA)
# Credit Card Fraud Dataset
# ==========================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# ==========================================
# 1. Check dataset information
# ==========================================

print("Dataset Shape:", df.shape)

print("\nDataset Information:")
df.info()

# ==========================================
# 2. Calculate legitimate and fraudulent
#    transactions
# ==========================================

legitimate_count = (df["fraud"] == 0).sum()
fraudulent_count = (df["fraud"] == 1).sum()

total_transactions = len(df)

print("\n===== Transaction Counts =====")
print("Number of legitimate transactions:", legitimate_count)
```

```
print("Number of fraudulent transactions:", fraudulent_count)
print("Total transactions:", total_transactions)

# ==========================================
# 3. Calculate class percentages
# ==========================================

legitimate_percentage = (legitimate_count / total_transactions) * 100
fraudulent_percentage = (fraudulent_count / total_transactions) * 100

print("\n===== Class Percentages =====")
print(f"Percentage of legitimate transactions: {legitimate_percentage:.2f}%"
print(f"Percentage of fraudulent transactions: {fraudulent_percentage:.2f}%"

# ==========================================
# 4. Display class distribution table
# ==========================================

class_distribution = pd.DataFrame({
    "Class": ["Legitimate (0)", "Fraudulent (1)"],
    "Number of Transactions": [
        legitimate_count,
        fraudulent_count
    ],
    "Percentage": [
        legitimate_percentage,
        fraudulent_percentage
    ]
})

print("\n===== Class Distribution =====")
display(class_distribution)

# ==========================================
# 5. Distribution of numerical features
# ==========================================

numerical_features = df.select_dtypes(
    include=np.number
).columns.tolist()

print("\nNumerical Features:")
print(numerical_features)

print("\nDescriptive Statistics:")
display(df[numerical_features].describe())

# ==========================================
# 6. Histograms of numerical features
# ==========================================

df[numerical_features].hist(
    figsize=(15, 12),
    bins=30
)
```

```
plt.suptitle(
    "Distribution of Numerical Features",
    fontsize=16
)

plt.tight_layout()
plt.show()

# ==========================================
# 7. Class distribution - Bar Chart
# ==========================================

plt.figure(figsize=(7, 5))

class_counts = df["fraud"].value_counts().sort_index()

plt.bar(
    ["Legitimate (0)", "Fraudulent (1)"],
    class_counts.values
)

plt.xlabel("Transaction Class")
plt.ylabel("Number of Transactions")
plt.title("Class Distribution of Credit Card Transactions")

# Display values above bars
for i, value in enumerate(class_counts.values):
    plt.text(
        i,
        value,
        f"{value:,}",
        ha="center",
        va="bottom"
    )

plt.show()

# ==========================================
# 8. Class distribution - Percentage Pie Chart
# ==========================================

plt.figure(figsize=(7, 7))

plt.pie(
    [legitimate_count, fraudulent_count],
    labels=["Legitimate (0)", "Fraudulent (1)"],
    autopct="%1.2f%%",
    startangle=90
)

plt.title("Percentage Distribution of Credit Card Transactions")
plt.show()

# ==========================================
```

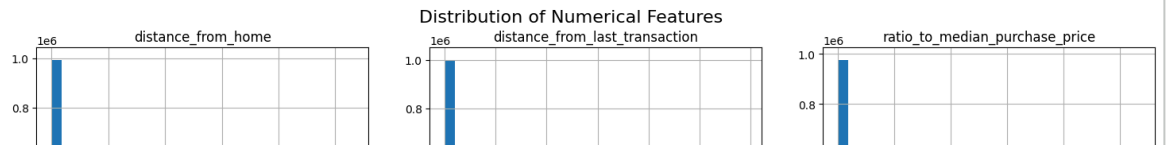
```
# 9. Check imbalance ratio
# ============================================

imbalance_ratio = legitimate_count / fraudulent_count

print("\n===== Class Imbalance Analysis =====")
print(
    f"There are approximately {imbalance_ratio:.2f} "
    "legitimate transactions for every fraudulent transaction."
)
```

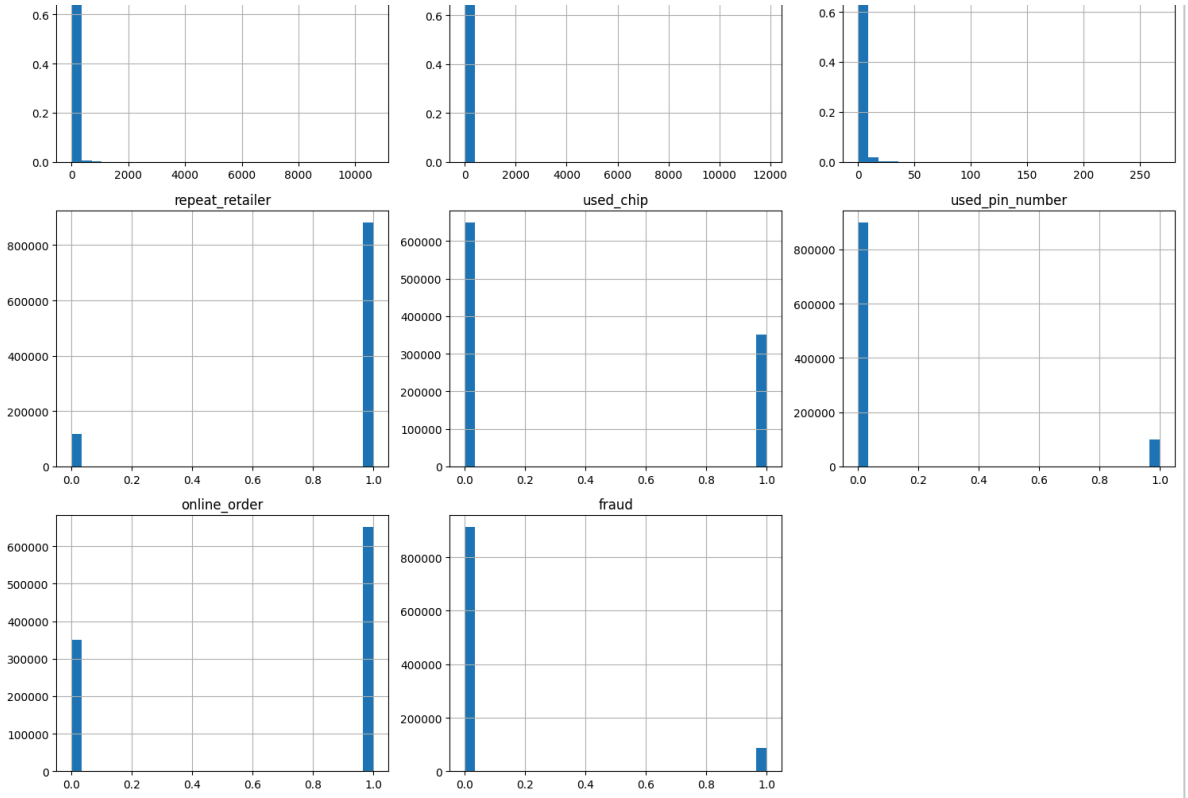
```
Dataset Shape: (1000000, 8)

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 8 columns):
 #   Column                          Non-Null Count    Dtype
---  ------                          --------------    -----
 0   distance_from_home              1000000 non-null  float64
 1   distance_from_last_transaction  1000000 non-null  float64
 2   ratio_to_median_purchase_price  1000000 non-null  float64
 3   repeat_retailer                 1000000 non-null  float64
 4   used_chip                       1000000 non-null  float64
 5   used_pin_number                 1000000 non-null  float64
 6   online_order                    1000000 non-null  float64
 7   fraud                           1000000 non-null  float64
dtypes: float64(8)
memory usage: 61.0 MB

===== Transaction Counts =====
Number of legitimate transactions: 912597
Number of fraudulent transactions: 87403
Total transactions: 1000000

===== Class Percentages =====
Percentage of legitimate transactions: 91.26%
Percentage of fraudulent transactions: 8.74%

===== Class Distribution =====
```

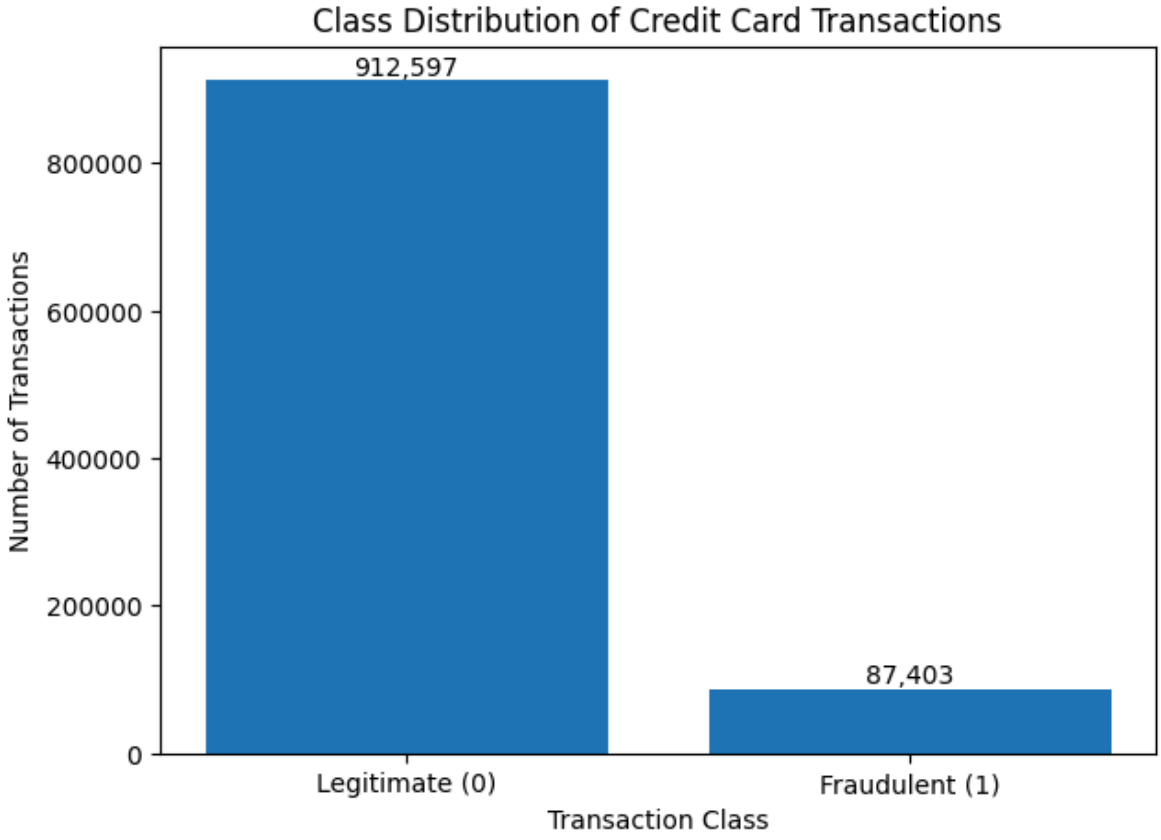
	Class	Number of Transactions	Percentage
0	Legitimate (0)	912597	91.2597
1	Fraudulent (1)	87403	8.7403

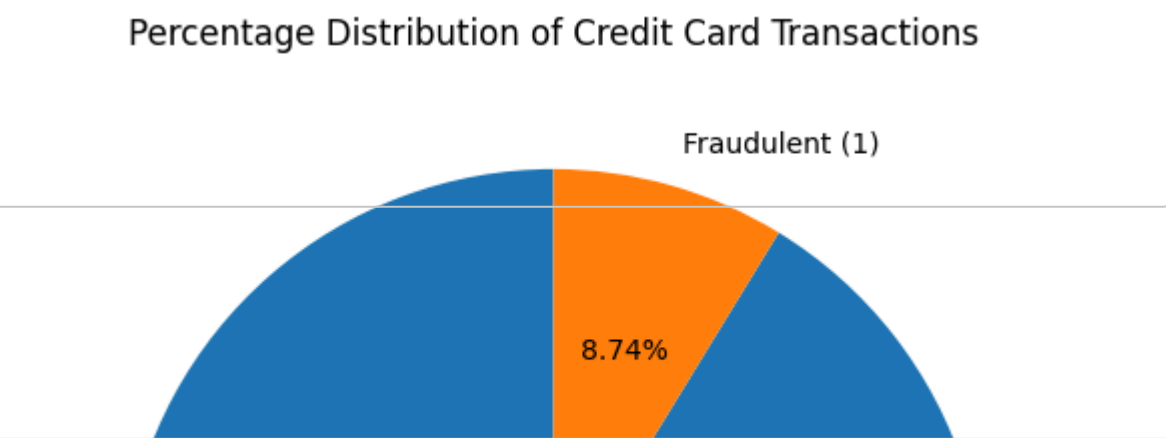
```
Numerical Features:
['distance_from_home', 'distance_from_last_transaction', 'ratio_to_median_pur

Descriptive Statistics:
```

	distance_from_home	distance_from_last_transaction	ratio_to_median_pu
count	1000000.000000	1000000.000000	1
mean	26.628792	5.036519	
std	65.390784	25.843093	
min	0.004874	0.000118	
25%	3.878008	0.296671	
50%	9.967760	0.998650	
75%	25.743985	3.355748	
max	10632.723672	11851.104565	

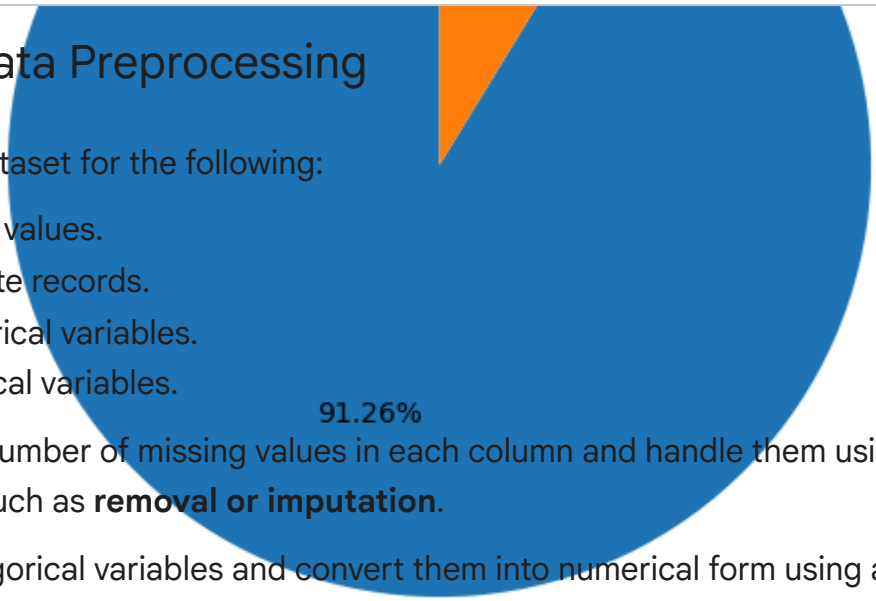



repeat_retailer
used_chip
used_pin_number
online_order
fraud


Class Distribution of Credit Card Transactions
912,597
87,403
Number of Transactions
Legitimate (0)
Fraudulent (1)
Transaction Class


Percentage Distribution of Credit Card Transactions
Fraudulent (1)
8.74%

Task 3: Data Preprocessing



Check the dataset for the following:

- Missing values.
- Duplicate records.
- Categorical variables.
- Numerical variables.

Identify the number of missing values in each column and handle them using suitable techniques such as **removal or imputation**.

Identify categorical variables and convert them into numerical form using appropriate **encoding techniques**.

For example, categorical variables can be converted into numerical values using **One-Hot Encoding**.

```
===== Class Imbalance Analysis =====
There are approximately 10744 legitimate transactions for every fraudulent tr
```

Also, identify the numerical features that will be used for building the Logistic Regression model.

The objective of this task is to prepare a clean and suitable dataset for further processing, feature scaling, model training, and evaluation.

```
# ============================================
# Task 3: Data Preprocessing
# Credit Card Fraud Detection
# ============================================

import pandas as pd
import numpy as np

# ============================================
# 1. Dataset overview
# ============================================

print("Original Dataset Shape:", df.shape)

print("\nFirst 5 records:")
display(df.head())

# ============================================
# 2. Check missing values
# ============================================

print("\n===== Missing Values =====")

missing_values = df.isnull().sum()

missing_table = pd.DataFrame({
    "Column": df.columns,
```

```
    "Missing Values": missing_values.values,
    "Missing Percentage": (
        missing_values.values / len(df) * 100
    )
})

display(missing_table)

# ============================================
# 3. Handle missing values
# ============================================

# Separate numerical and categorical columns
numerical_columns = df.select_dtypes(
    include=np.number
).columns.tolist()

categorical_columns = df.select_dtypes(
    include=["object", "category", "bool"]
).columns.tolist()

# Remove target variable from feature lists
if "fraud" in numerical_columns:
    numerical_columns.remove("fraud")

if "fraud" in categorical_columns:
    categorical_columns.remove("fraud")

print("\nNumerical columns:")
print(numerical_columns)

print("\nCategorical columns:")
print(categorical_columns)

# Fill numerical missing values using median
for column in numerical_columns:
    if df[column].isnull().sum() > 0:
        df[column] = df[column].fillna(df[column].median())

# Fill categorical missing values using mode
for column in categorical_columns:
    if df[column].isnull().sum() > 0:
        df[column] = df[column].fillna(df[column].mode()[0])

print("\nMissing values after handling:")
print(df.isnull().sum())

# ============================================
# 4. Check duplicate records
# ============================================

duplicate_count = df.duplicated().sum()

print("\n===== Duplicate Records =====")
print("Number of duplicate records:", duplicate_count)
```

```
# Remove duplicate records
if duplicate_count > 0:
    df = df.drop_duplicates().reset_index(drop=True)
    print("Duplicate records removed.")
else:
    print("No duplicate records found.")

print("Dataset shape after removing duplicates:", df.shape)

# ============================================
# 5. Identify categorical variables
# ============================================

categorical_columns = df.select_dtypes(
    include=["object", "category", "bool"]
).columns.tolist()

# Target should not be encoded as a feature
if "fraud" in categorical_columns:
    categorical_columns.remove("fraud")

print("\n===== Categorical Variables =====")

if len(categorical_columns) == 0:
    print("No categorical variables found.")
else:
    print(categorical_columns)

# ============================================
# 6. Identify numerical variables
# ============================================

numerical_columns = df.select_dtypes(
    include=np.number
).columns.tolist()

if "fraud" in numerical_columns:
    numerical_columns.remove("fraud")

print("\n===== Numerical Variables =====")
print(numerical_columns)

# ============================================
# 7. One-Hot Encoding
# ============================================

if len(categorical_columns) > 0:

    print("\nApplying One-Hot Encoding...")

    df_encoded = pd.get_dummies(
        df,
        columns=categorical_columns,
        drop_first=True,
```

```
        dtype=int
    )

else:

    print("\nNo categorical variables found.")
    df_encoded = df.copy()

print("\nDataset after encoding:")
display(df_encoded.head())

print("\nShape after encoding:", df_encoded.shape)

# ============================================
# 8. Separate features and target
# ============================================

X = df_encoded.drop(columns=["fraud"])
y = df_encoded["fraud"]

print("\n===== Features and Target =====")

print("Feature matrix X shape:", X.shape)
print("Target vector y shape:", y.shape)

print("\nFeatures used for Logistic Regression:")
print(X.columns.tolist())

print("\nTarget variable:")
print("fraud")

# ============================================
# 9. Check final data types
# ============================================

print("\n===== Final Data Types =====")
print(X.dtypes)

# ============================================
# 10. Final missing-value check
# ============================================

print("\n===== Final Missing Value Check =====")
print("Missing values in X:", X.isnull().sum().sum())
print("Missing values in y:", y.isnull().sum())

# ============================================
# 11. Final dataset summary
# ============================================

print("\n===== Final Dataset Summary =====")
print("Number of samples:", X.shape[0])
print("Number of features:", X.shape[1])
print("Legitimate transactions:", (y == 0).sum())
print("Fraudulent transactions:", (y == 1).sum())
```

```
print("\nTask 3 preprocessing completed successfully!")
```

```
Original Dataset Shape: (1000000, 8)

First 5 records:
```

	distance_from_home	distance_from_last_transaction	ratio_to_median_purcha
0	57.877857	0.311140	
1	10.829943	0.175592	
2	5.091079	0.805153	
3	2.247564	5.600044	
4	44.190936	0.566486	

```
===== Missing Values =====
```

	Column	Missing Values	Missing Percentage
0	distance_from_home	0	0.0
1	distance_from_last_transaction	0	0.0
2	ratio_to_median_purchase_price	0	0.0
3	repeat_retailer	0	0.0
4	used_chip	0	0.0
5	used_pin_number	0	0.0
6	online_order	0	0.0
7	fraud	0	0.0

```
Numerical columns:
['distance_from_home', 'distance_from_last_transaction', 'ratio_to_median_pur

Categorical columns:
[]

Missing values after handling:
distance_from_home                0
distance_from_last_transaction    0
ratio_to_median_purchase_price    0
repeat_retailer                   0
used_chip                         0
used_pin_number                   0
online_order                      0
fraud                             0
dtype: int64

===== Duplicate Records =====
Number of duplicate records: 0
No duplicate records found.
Dataset shape after removing duplicates: (1000000, 8)
===== Categorical Variables =====
No categorical variables found.
===== Numerical Variables =====
['distance_from_home', 'distance_from_last_transaction', 'ratio_to_median_pur
```

Task 4: Feature Scaling and Class Imbalance Handling

Standardize the numerical features using **Standardization** so that they have approximately zero mean and unit variance.

The standardized value of a feature is calculated as:

```
No categorical variables found.
```

$$z = \frac{x - \mu}{\sigma}$$

where:

- (x) = original feature value
- (\mu) = mean of the feature
- (\sigma) = standard deviation of the feature
- (z) = standardized feature value

Handle the **class imbalance** in the dataset using a suitable technique such as:

- Random Undersampling
- Oversampling
- Class Weighting

Display the class distribution **before and after** applying the selected class-imbalance handling technique.

Compare the number of legitimate and fraudulent transactions before and after balancing.

The objective of this task is to ensure that the features are on a comparable scale and that the Logistic Regression model does not become biased toward the majority class.

```
Dataset after encoding:
```

	distance_from_home	distance_from_last_transaction	ratio_to_median_purcha
0	57.877857	0.311140	
1	10.829943	0.175592	
2	5.091079	0.805153	
3	2.247564	5.600044	
4	44.190936	0.566486	

```
Shape after encoding: (1000000, 8)
==== Features and Target ====
Feature matrix X shape: (1000000, 7)
Target vector y shape: (1000000,)
Features used for Logistic Regression:
['distance_from_home', 'distance_from_last_transaction', 'ratio_to_median_pur
Target variable:
fraud
===== Final Data Types =====
distance_from_home               float64
```

```
# ============================================
# Task 4: Feature Scaling and Class Imbalance
# Credit Card Fraud Detection
# ============================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.utils.class_weight import compute_class_weight

# ============================================
# 1. Display original class distribution
# ============================================

print("===== CLASS DISTRIBUTION BEFORE HANDLING =====")

class_counts_before = y.value_counts().sort_index()

legitimate_before = class_counts_before.get(0, 0)
fraudulent_before = class_counts_before.get(1, 0)

print("Legitimate transactions:", legitimate_before)
print("Fraudulent transactions:", fraudulent_before)
```

```
print("\nClass percentages before handling:")
print(
    f"Legitimate: "
    f"{legitimate_before / len(y) * 100:.2f}%"
)

print(
    f"Fraudulent: "
    f"{fraudulent_before / len(y) * 100:.2f}%"
)

# ============================================
# 2. Train-Test Split
# ============================================

# Stratification keeps the same fraud ratio
# in both training and testing data.

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\n===== TRAIN-TEST SPLIT =====")
print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])

# ============================================
# 3. Feature Scaling
# ============================================

scaler = StandardScaler()

# Fit scaler ONLY on training data
X_train_scaled = scaler.fit_transform(X_train)

# Use the same scaler on test data
X_test_scaled = scaler.transform(X_test)

# Convert back to DataFrame
X_train_scaled = pd.DataFrame(
    X_train_scaled,
    columns=X_train.columns,
    index=X_train.index
)

X_test_scaled = pd.DataFrame(
    X_test_scaled,
    columns=X_test.columns,
    index=X_test.index
)
```

```
print("\n===== FEATURE SCALING =====")

print("Mean of standardized training features:")
print(X_train_scaled.mean().round(4))

print("\nStandard deviation of standardized training features:")
print(X_train_scaled.std().round(4))

# ============================================
# 4. Verify Standardization
# ============================================

print("\nFirst five standardized training records:")
display(X_train_scaled.head())

# ============================================
# 5. Handle Class Imbalance using Class Weighting
# ============================================

print("\n===== CLASS WEIGHTING =====")

classes = np.unique(y_train)

weights = compute_class_weight(
    class_weight="balanced",
    classes=classes,
    y=y_train
)

class_weights = dict(zip(classes, weights))

print("Calculated class weights:")
print(class_weights)

# ============================================
# 6. Display training class distribution
# ============================================

train_class_counts = y_train.value_counts().sort_index()

print("\nTraining class distribution:")
print(
    "Legitimate transactions:",
    train_class_counts.get(0, 0)
)

print(
    "Fraudulent transactions:",
    train_class_counts.get(1, 0)
)

# ============================================
# 7. Display class distribution comparison
# ============================================
```

```
comparison = pd.DataFrame({
    "Class": [
        "Legitimate (0)",
        "Fraudulent (1)"
    ],

    "Before Handling": [
        legitimate_before,
        fraudulent_before
    ],

    "After Class Weighting": [
        train_class_counts.get(0, 0),
        train_class_counts.get(1, 0)
    ]
})

print("\n===== CLASS DISTRIBUTION COMPARISON =====")
display(comparison)

# ============================================
# 8. Visualize class distribution
# ============================================

plt.figure(figsize=(10, 5))

x = np.arange(2)
width = 0.35

plt.bar(
    x - width / 2,
    [
        legitimate_before,
        fraudulent_before
    ],
    width,
    label="Before Handling"
)

plt.bar(
    x + width / 2,
    [
        train_class_counts.get(0, 0),
        train_class_counts.get(1, 0)
    ],
    width,
    label="Training Data"
)

plt.xticks(
    x,
    ["Legitimate (0)", "Fraudulent (1)"]
)
```

```
plt.ylabel("Number of Transactions")
plt.xlabel("Transaction Class")
plt.title("Class Distribution Before and After Class Weighting")
plt.legend()

plt.show()

# ============================================
# 9. Prepare data for Logistic Regression
# ============================================

print("\n===== FINAL DATA FOR MODEL TRAINING =====")

print("X_train_scaled shape:", X_train_scaled.shape)
print("X_test_scaled shape:", X_test_scaled.shape)

print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)

print("\nClass weights to use during Logistic Regression:")
print(class_weights)
```

```
===== CLASS DISTRIBUTION BEFORE HANDLING =====
Legitimate transactions: 912597
Fraudulent transactions: 87403

Class percentages before handling:
Legitimate: 91.26%
Fraudulent: 8.74%

===== TRAIN-TEST SPLIT =====
Training samples: 800000
Testing samples: 200000

===== FEATURE SCALING =====
Mean of standardized training features:
distance_from_home                -0.0
distance_from_last_transaction     0.0
ratio_to_median_purchase_price    -0.0
repeat_retailer                    0.0
used_chip                          0.0
used_pin_number                   -0.0
online_order                       0.0
dtype: float64

Standard deviation of standardized training features:
distance_from_home                 1.0
distance_from_last_transaction     1.0
ratio_to_median_purchase_price     1.0
repeat_retailer                    1.0
used_chip                          1.0
used_pin_number                    1.0
online_order                       1.0
dtype: float64

First five standardized training records:
```

	distance_from_home	distance_from_last_transaction	ratio_to_median_p
950236	-0.353541	0.083049	
382251	-0.255...	-0.156580	
597434	0.078467	-0.070319	
296537	-0.250188	-0.187940	
235768	0.839938	-0.010984	

```
===== CLASS WEIGHTING =====
Calculated class weights:
{np.float64(0.0): np.float64(0.5478866041046509), np.float64(1.0): np.float64
Training class distribution:
Legitimate transactions: 730078
Fraudulent transactions: 69922

===== CLASS DISTRIBUTION COMPARISON =====
```

	Class	Before Handling	After Class Weighting
0	Legitimate (0)	912597	730078
1	Fraudulent (1)	87403	69922

Task 5: Split the Dataset

Divide the preprocessed dataset into three subsets:

- **Training Set -- 60%**
- **Validation Set -- 20%**
- **Test Set -- 20%**

Use **stratified splitting** to maintain a representative proportion of fraudulent and legitimate transactions in each subset.

The dataset should first be divided into training and temporary sets. The temporary set should then be divided equally into validation and test sets.

The objective of this task is to use the training set for model training, the validation set for model evaluation and parameter selection, and the test set for final performance evaluation.

```
# ============================================
# Task 5: Split the Dataset
# 60% Training - 20% Validation - 20% Testing
# ============================================

from sklearn.model_selection import train_test_split

# ============================================
# 1. First Split
#    Training = 60%
#    Temporary = 40%
# ============================================

X_train, X_temp, y_train, y_temp = train_test_split(
    X,
    y,
    test_size=0.40,
    random_state=42,
    stratify=y
)

# ============================================
# 2. Second Split
#    Validation = 20%
#    Test = 20%
#
#    Temporary dataset = 40%
#    Split it equally into 20% + 20%
# ============================================

X_val, X_test, y_val, y_test = train_test_split(
    X_temp,
    y_temp,
    test_size=0.50,
    random_state=42,
    stratify=y_temp
)

# ============================================
# 3. Display dataset sizes
# ============================================

print("===== DATASET SPLIT =====")

print("\nTraining Set:")
print("Number of samples:", X_train.shape[0])
print("Percentage:", X_train.shape[0] / len(X) * 100, "%")

print("\nValidation Set:")
print("Number of samples:", X_val.shape[0])
print("Percentage:", X_val.shape[0] / len(X) * 100, "%")

print("\nTest Set:")
print("Number of samples:", X_test.shape[0])
```

```
print("Percentage:", X_test.shape[0] / len(X) * 100, "%")

# ============================================
# 4. Check target class distribution
# ============================================

print("\n===== CLASS DISTRIBUTION =====")

print("\nTraining Set:")
print(y_train.value_counts())
print(y_train.value_counts(normalize=True) * 100)

print("\nValidation Set:")
print(y_val.value_counts())
print(y_val.value_counts(normalize=True) * 100)

print("\nTest Set:")
print(y_test.value_counts())
print(y_test.value_counts(normalize=True) * 100)

# ============================================
# 5. Create comparison table
# ============================================

split_comparison = pd.DataFrame({
    "Dataset": ["Training", "Validation", "Test"],
    "Samples": [
        len(y_train),
        len(y_val),
        len(y_test)
    ],
    "Legitimate (0)": [
        (y_train == 0).sum(),
        (y_val == 0).sum(),
        (y_test == 0).sum()
    ],
    "Fraudulent (1)": [
        (y_train == 1).sum(),
        (y_val == 1).sum(),
        (y_test == 1).sum()
    ]
})

# Calculate percentages
split_comparison["Fraud Percentage"] = (
    split_comparison["Fraudulent (1)"]
    / split_comparison["Samples"]
    * 100
)

print("\n===== SPLIT COMPARISON =====")
display(split_comparison)

# ============================================
# 6. Verify total number of samples
```

```
# ============================================

print("\n===== VERIFICATION =====")

total_split = (
    len(X_train) +
    len(X_val) +
    len(X_test)
)

print("Original dataset samples:", len(X))
print("Total after splitting:", total_split)

if total_split == len(X):
    print("✓ All samples are correctly distributed.")
else:
    print("✗ Sample count mismatch.")

# ============================================
# 7. Verify stratification
# ============================================

print("\n===== FRAUD CLASS PERCENTAGES =====")

print(
    f"Training fraud percentage: "
    f"{(y_train == 1).mean() * 100:.2f}%"
)

print(
    f"Validation fraud percentage: "
    f"{(y_val == 1).mean() * 100:.2f}%"
)

print(
    f"Test fraud percentage: "
    f"{(y_test == 1).mean() * 100:.2f}%"
)
```

```
===== DATASET SPLIT =====

Training Set:
Number of samples: 600000
Percentage: 60.0 %

Validation Set:
Number of samples: 200000
Percentage: 20.0 %

Test Set:
Number of samples: 200000
Percentage: 20.0 %

===== CLASS DISTRIBUTION =====

Training Set:
fraud
0.0    547558
1.0     52442
Name: count, dtype: int64
fraud
0.0    91.259667
1.0     8.740333
Name: proportion, dtype: float64

Validation Set:
fraud
0.0    182520
1.0     17481
Name: count, dtype: int64
fraud
0.0    91.2595
1.0     8.7405
Name: proportion, dtype: float64

Test Set:
fraud
0.0    182520
1.0     17480
Name: count, dtype: int64
fraud
0.0    91.26
1.0     8.74
Name: proportion, dtype: float64
```

Task 6: Implement Sigmoid and Cost Functions

Implement the **Sigmoid Function** used in Logistic Regression.

The Sigmoid Function is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The Sigmoid Function converts the output of the linear equation into a probability value between 0 and 1.

Implement the **Binary Cross-Entropy Cost Function**:

$$J(w, b) = -\frac{1}{m} \sum_{i=1}^{m} [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

where:

- (m) = number of training samples
- (y_i) = actual class label
- (\hat{y}_i) = predicted probability
- (w) = weight vector
- (b) = bias

Test the Sigmoid Function using suitable sample values.

Calculate the initial cost of the Logistic Regression model using the initialized weights and bias.

The objective of this task is to understand how the Sigmoid Function produces probabilities and how the Binary Cross-Entropy Cost Function measures the error of the Logistic Regression model.

```
# ==========================================
# Task 6: Sigmoid and Cost Functions
# Logistic Regression
# ==========================================

import numpy as np

# ==========================================
# 1. Define Sigmoid Function
# ==========================================

def sigmoid(z):
    """
```

```
    Sigmoid function:
        sigma(z) = 1 / (1 + e^(-z))
    """

    # Clip values to prevent overflow
    z = np.clip(z, -500, 500)

    return 1 / (1 + np.exp(-z))


# ============================================
# 2. Test Sigmoid Function
# ============================================

sample_values = np.array([
    -10,
    -5,
    -2,
    -1,
    0,
    1,
    2,
    5,
    10
])

sigmoid_values = sigmoid(sample_values)

print("===== SIGMOID FUNCTION TEST =====")

for z, result in zip(sample_values, sigmoid_values):
    print(f"z = {z:>3}  -->  sigmoid(z) = {result:.6f}")


# ============================================
# 3. Verify sigmoid output range
# ============================================

print("\n===== SIGMOID OUTPUT RANGE =====")

print("Minimum output:", sigmoid_values.min())
print("Maximum output:", sigmoid_values.max())

print("\nAll outputs are between 0 and 1:",
      np.all((sigmoid_values >= 0) &
             (sigmoid_values <= 1)))


# ============================================
# 4. Define Binary Cross-Entropy Cost Function
# ============================================

def compute_cost(X, y, w, b):
    """
    Binary Cross-Entropy Cost:
```

```
    J(w,b) = -1/m * sum[
        y log(y_hat) +
        (1-y) log(1-y_hat)
    ]
    """

    m = X.shape[0]

    # Linear equation
    z = np.dot(X, w) + b

    # Predicted probabilities
    y_hat = sigmoid(z)

    # Avoid log(0)
    epsilon = 1e-15

    y_hat = np.clip(
        y_hat,
        epsilon,
        1 - epsilon
    )

    # Binary Cross-Entropy Cost
    cost = -(1 / m) * np.sum(
        y * np.log(y_hat) +
        (1 - y) * np.log(1 - y_hat)
    )

    return cost


# ============================================
# 5. Initialize weights and bias
# ============================================

# Number of features
n_features = X_train.shape[1]

# Initialize weights with zeros
w_initial = np.zeros(n_features)

# Initialize bias with zero
b_initial = 0.0

print("\n===== INITIAL PARAMETERS =====")

print("Number of features:", n_features)
print("Initial bias:", b_initial)
print("Initial weights:")
print(w_initial)


# ============================================
```

```
# 6. Calculate initial predicted probabilities
# ============================================

z_initial = np.dot(
    X_train,
    w_initial
) + b_initial

y_pred_initial = sigmoid(z_initial)

print("\n===== INITIAL PREDICTIONS =====")

print("First 10 predicted probabilities:")

print(y_pred_initial[:10])


# ============================================
# 7. Calculate Initial Cost
# ============================================

initial_cost = compute_cost(
    X_train,
    y_train.to_numpy(),
    w_initial,
    b_initial
)

print("\n===== INITIAL COST =====")

print(
    f"Initial Binary Cross-Entropy Cost: "
    f"{initial_cost:.6f}"
)


# ============================================
# 8. Verify theoretical initial cost
# ============================================

print("\n===== COST VERIFICATION =====")

print(
    "When w = 0 and b = 0:"
)

print(
    "z = 0 for every training sample"
)

print(
    "sigmoid(0) = 0.5"
)

print(
```

```
    "Therefore, the model initially predicts "
    "P(fraud) = 0.5 for every transaction."
)

print(
    f"\nCalculated initial cost = {initial_cost:.6f}"
)
```

```
===== SIGMOID FUNCTION TEST =====
z = -10  -->  sigmoid(z) = 0.000045
z =  -5  -->  sigmoid(z) = 0.006693
z =  -2  -->  sigmoid(z) = 0.119203
z =  -1  -->  sigmoid(z) = 0.268941
z =   0  -->  sigmoid(z) = 0.500000
z =   1  -->  sigmoid(z) = 0.731059
z =   2  -->  sigmoid(z) = 0.880797
z =   5  -->  sigmoid(z) = 0.993307
z =  10  -->  sigmoid(z) = 0.999955

===== SIGMOID OUTPUT RANGE =====
Minimum output: 4.5397868702434395e-05
Maximum output: 0.9999546021312976

All outputs are between 0 and 1: True

===== INITIAL PARAMETERS =====
Number of features: 7
Initial bias: 0.0
Initial weights:
[0. 0. 0. 0. 0. 0. 0.]

===== INITIAL PREDICTIONS =====
First 10 predicted probabilities:
[0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5]

===== INITIAL COST =====
Initial Binary Cross-Entropy Cost: 0.693147

===== COST VERIFICATION =====
When w = 0 and b = 0:
z = 0 for every training sample
sigmoid(0) = 0.5
Therefore, the model initially predicts P(fraud) = 0.5 for every transaction.

Calculated initial cost = 0.693147
```

Task 7: Implement Logistic Regression Using Gradient Descent

Implement **Logistic Regression from scratch** using NumPy without directly using the Scikit-learn Logistic Regression implementation.

Initialize the model parameters, including the **weight vector** (w) and **bias** (b).

Calculate the predicted probability using the Sigmoid Function:

$$\hat{y} = \sigma(Xw + b)$$

where:

- (X) = input feature matrix
- (w) = weight vector
- $\hat{y} = \text{predicted probability}$

Calculate the gradients of the cost function with respect to the weights and bias:

$$\frac{\partial J}{\partial w} = \frac{1}{m} X^T (\hat{y} - y)$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}_i - y_i)$$

Update the model parameters using **Gradient Descent**:

$$w := w - \alpha \frac{\partial J}{\partial w}$$

$$b := b - \alpha \frac{\partial J}{\partial b}$$

where (α) represents the **learning rate**.

Train the model for a suitable number of iterations and record the cost value during each iteration.

Plot **Cost vs. Iteration** to verify whether the optimization process is converging.

The objective of this task is to implement Logistic Regression manually and understand how Gradient Descent optimizes the model parameters by minimizing the Binary Cross-Entropy Cost Function.

```
# ==========================================
# Task 7: Logistic Regression Using
#         Gradient Descent
# ==========================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


# ==========================================
# 1. Sigmoid Function
# ==========================================

def sigmoid(z):
    """
    Sigmoid function:

        sigma(z) = 1 / (1 + e^(-z))
```

```
    """

    # Prevent numerical overflow
    z = np.clip(z, -500, 500)

    return 1 / (1 + np.exp(-z))


# ============================================
# 2. Binary Cross-Entropy Cost Function
# ============================================

def compute_cost(X, y, w, b):
    """
    Binary Cross-Entropy Cost:

    J(w,b) = -1/m * sum[
        y*log(y_hat) +
        (1-y)*log(1-y_hat)
    ]
    """

    m = X.shape[0]

    # Linear prediction
    z = np.dot(X, w) + b

    # Predicted probability
    y_hat = sigmoid(z)

    # Prevent log(0)
    epsilon = 1e-15

    y_hat = np.clip(
        y_hat,
        epsilon,
        1 - epsilon
    )

    # Binary Cross-Entropy
    cost = -(1 / m) * np.sum(
        y * np.log(y_hat) +
        (1 - y) * np.log(1 - y_hat)
    )

    return cost


# ============================================
# 3. Gradient Descent Function
# ============================================

def gradient_descent(
    X,
    y,
```

```
    learning_rate=0.01,
    iterations=1000
):
    """
    Train Logistic Regression using
    Gradient Descent.
    """

    # Number of training examples
    m = X.shape[0]

    # Number of features
    n_features = X.shape[1]

    # Initialize weights
    w = np.zeros(n_features)

    # Initialize bias
    b = 0.0

    # Store cost values
    cost_history = []

    # ========================================
    # Gradient Descent Iterations
    # ========================================

    for i in range(iterations):

        # ----------------------------------
        # Step 1: Calculate linear prediction
        # ----------------------------------

        z = np.dot(X, w) + b

        # ----------------------------------
        # Step 2: Calculate predicted
        #         probabilities
        # ----------------------------------

        y_hat = sigmoid(z)

        # ----------------------------------
        # Step 3: Calculate gradients
        # ----------------------------------

        error = y_hat - y

        dw = (1 / m) * np.dot(X.T, error)

        db = (1 / m) * np.sum(error)

        # ----------------------------------
        # Step 4: Update weights and bias
        # ----------------------------------
```

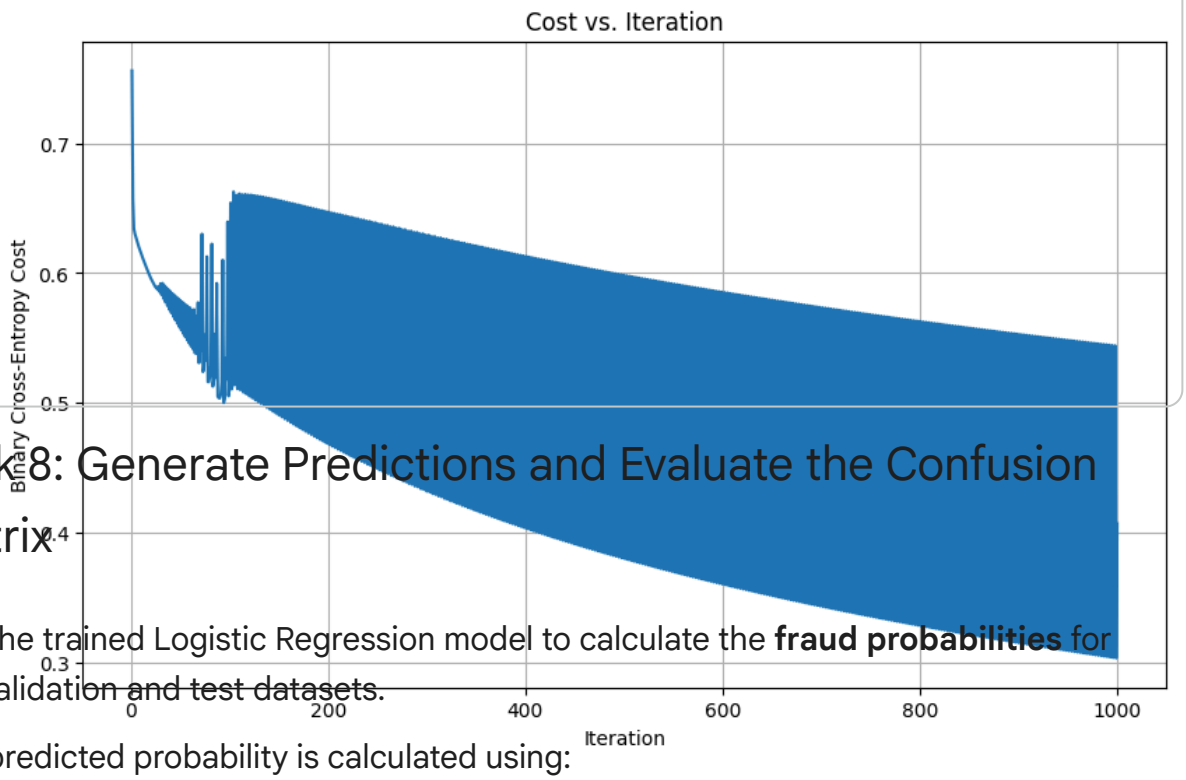
```
        w = w - learning_rate * dw

        b = b - learning_rate * db

        # -----------------------------------
        # Step 5: Calculate and store cost
        # -----------------------------------

        cost = compute_cost(
            X,
            y,
            w,
            b
        )

        cost_history.append(cost)

        # Display progress
        if i % 100 == 0:
            print(
                f"Iteration {i:4d} | "
                f"Cost = {cost:.6f}"
            )

    return w, b, cost_history


# ==========================================
# 4. Convert training and validation data
#    to NumPy arrays
# ==========================================

X_train_np = np.asarray(X_train, dtype=float)

X_val_np = np.asarray(X_val, dtype=float)

y_train_np = np.asarray(y_train, dtype=float)

y_val_np = np.asarray(y_val, dtype=float)


# ==========================================
# 5. Set Hyperparameters
# ==========================================

learning_rate = 0.01
iterations = 1000

print("===== HYPERPARAMETERS =====")
print("Learning rate:", learning_rate)
print("Number of iterations:", iterations)


# ==========================================
```

```
# 6. Train Logistic Regression
# ============================================

print("\n===== TRAINING MODEL =====")

w, b, cost_history = gradient_descent(
    X_train_np,
    y_train_np,
    learning_rate=learning_rate,
    iterations=iterations
)


# ============================================
# 7. Display final parameters
# ============================================

print("\n===== FINAL PARAMETERS =====")

print("Final bias:")
print(b)

print("\nFinal weights:")
print(w)


# ============================================
# 8. Display initial and final cost
# ============================================

print("\n===== COST =====")

print(
    f"Initial cost: {cost_history[0]:.6f}"
)

print(
    f"Final cost: {cost_history[-1]:.6f}"
)


# ============================================
# 9. Plot Cost vs Iteration
# ============================================

plt.figure(figsize=(10, 6))

plt.plot(
    range(1, iterations + 1),
    cost_history
)

plt.xlabel("Iteration")
plt.ylabel("Binary Cross-Entropy Cost")
plt.title("Cost vs. Iteration")
```

```
plt.grid(True)

plt.show()


# ==========================================
# 10. Prediction Function
# ==========================================

def predict_probability(X, w, b):
    """
    Calculate probability of fraud.
    """

    z = np.dot(X, w) + b

    return sigmoid(z)


def predict_class(X, w, b, threshold=0.5):
    """
    Convert probability into class label.
    """

    probabilities = predict_probability(
        X,
        w,
        b
    )

    return (probabilities >= threshold).astype(int)


# ==========================================
# 11. Make predictions on validation set
# ==========================================

y_val_probability = predict_probability(
    X_val_np,
    w,
    b
)

y_val_prediction = predict_class(
    X_val_np,
    w,
    b,
    threshold=0.5
)


# ==========================================
# 12. Display sample predictions
# ==========================================
```

```
prediction_table = pd.DataFrame({
    "Actual": y_val_np[:20].astype(int),
    "Predicted Probability": y_val_probability[:20],
    "Predicted Class": y_val_prediction[:20]
})

print("\n===== SAMPLE VALIDATION PREDICTIONS =====")

display(prediction_table)
```

```
===== HYPERPARAMETERS =====
Learning rate: 0.01
Number of iterations: 1000

===== TRAINING MODEL =====
Iteration    0 | Cost = 0.756115
Iteration  100 | Cost = 0.654245
Iteration  200 | Cost = 0.467846
Iteration  300 | Cost = 0.444763
Iteration  400 | Cost = 0.612973
Iteration  500 | Cost = 0.379624
Iteration  600 | Cost = 0.414113
Iteration  700 | Cost = 0.573515
Iteration  800 | Cost = 0.327932
Iteration  900 | Cost = 0.407659

===== FINAL PARAMETERS =====
Final bias:
-1.1709513313406703

Final weights:
[ 0.03003612  0.00759982  0.23753768 -0.90886688 -0.42091082 -0.19127705
 -0.41423144]

===== COST =====
Initial cost: 0.756115
Final cost: 0.406744
```



Task 8: Generate Predictions and Evaluate the Confusion Matrix

Use the trained Logistic Regression model to calculate the **fraud probabilities** for the validation and test datasets.

The predicted probability is calculated using:

$$\hat{y} = \sigma(Xw + b)$$

```
===== SAMPLE VALIDATION PREDICTIONS =====
```

	Actual	Predicted Probability	Predicted Class
0	0	0.215482	0
1	0	0.077170	0
2	0	0.167803	0
3	0	0.093226	0

Convert the predicted probabilities into class labels using a threshold of **0.5**:

$$\hat{y}_{class} = \begin{cases} 1, & \text{if } \hat{y} \geq 0.5 \\ 0, & \text{if } \hat{y} < 0.5 \end{cases}$$

Construct the **Confusion Matrix** and identify the following:

- **True Positive (TP):** Fraudulent transactions correctly classified as fraudulent.

- **True Negative (TN):** Legitimate transactions correctly classified as legitimate.
- **False Positive (FP):** Legitimate transactions incorrectly classified as fraudulent.
- **False Negative (FN):** Fraudulent transactions incorrectly classified as legitimate.

The confusion matrix can be represented as:

$$\text{Confusion Matrix} = \begin{bmatrix} TN & FP \\ FN & TP \end{bmatrix}$$

Explain the meaning of each result in the context of **credit card fraud detection**.

The objective of this task is to evaluate the classification results of the Logistic Regression model and understand the different types of prediction errors, with particular attention to **False Negatives**, which represent fraudulent transactions that are not detected.

4	0	0.082598	0
5	0	0.101799	0
6	0	0.160430	0
7	0	0.872360	1
8	0	0.129650	0
9	0	0.246930	0
10	0	0.820061	1
11	0	0.143056	0
12	0	0.104707	0
13	0	0.643820	1
14	0	0.999995	1

```
# ==========================================
# Task 8: Predictions and Confusion Matrix
# Logistic Regression - Fraud Detection
# ==========================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay


# ==========================================
# 1. Prediction Functions
# ==========================================

def predict_probability(X, w, b):
    """
    Calculate fraud probability:

        y_hat = sigmoid(Xw + b)
    """

    z = np.dot(X, w) + b

    return sigmoid(z)


def predict_class(X, w, b, threshold=0.5):
    """
    Convert probability into class:

        1 if probability >= threshold
        0 if probability < threshold
    """
```

```
    probabilities = predict_probability(X, w, b)

    predictions = (probabilities >= threshold).astype(int)

    return predictions


# ============================================
# 2. Convert datasets to NumPy arrays
# ============================================

X_val_np = np.asarray(X_val, dtype=float)
X_test_np = np.asarray(X_test, dtype=float)

y_val_np = np.asarray(y_val, dtype=int)
y_test_np = np.asarray(y_test, dtype=int)


# ============================================
# 3. Generate Validation Predictions
# ============================================

threshold = 0.5

y_val_probability = predict_probability(
    X_val_np,
    w,
    b
)

y_val_pred = predict_class(
    X_val_np,
    w,
    b,
    threshold
)


# ============================================
# 4. Generate Test Predictions
# ============================================

y_test_probability = predict_probability(
    X_test_np,
    w,
    b
)

y_test_pred = predict_class(
    X_test_np,
    w,
    b,
    threshold
)
```

```
# ==========================================
# 5. Display Sample Predictions
# ==========================================

print("===== VALIDATION PREDICTIONS =====")

validation_results = pd.DataFrame({
    "Actual Class": y_val_np[:20],
    "Fraud Probability": y_val_probability[:20],
    "Predicted Class": y_val_pred[:20]
})

display(validation_results)


print("\n===== TEST PREDICTIONS =====")

test_results = pd.DataFrame({
    "Actual Class": y_test_np[:20],
    "Fraud Probability": y_test_probability[:20],
    "Predicted Class": y_test_pred[:20]
})

display(test_results)


# ==========================================
# 6. Validation Confusion Matrix
# ==========================================

cm_val = confusion_matrix(
    y_val_np,
    y_val_pred
)

TN_val, FP_val, FN_val, TP_val = cm_val.ravel()

print("\n===== VALIDATION CONFUSION MATRIX =====")

print(cm_val)

print("\nTN:", TN_val)
print("FP:", FP_val)
print("FN:", FN_val)
print("TP:", TP_val)


# ==========================================
# 7. Test Confusion Matrix
# ==========================================

cm_test = confusion_matrix(
    y_test_np,
```

```
    y_test_pred
)

TN_test, FP_test, FN_test, TP_test = cm_test.ravel()

print("\n===== TEST CONFUSION MATRIX =====")

print(cm_test)

print("\nTN:", TN_test)
print("FP:", FP_test)
print("FN:", FN_test)
print("TP:", TP_test)


# ============================================
# 8. Display Validation Confusion Matrix
# ============================================

disp_val = ConfusionMatrixDisplay(
    confusion_matrix=cm_val,
    display_labels=[
        "Legitimate (0)",
        "Fraudulent (1)"
    ]
)

disp_val.plot()

plt.title("Validation Confusion Matrix")
plt.show()


# ============================================
# 9. Display Test Confusion Matrix
# ============================================

disp_test = ConfusionMatrixDisplay(
    confusion_matrix=cm_test,
    display_labels=[
        "Legitimate (0)",
        "Fraudulent (1)"
    ]
)

disp_test.plot()

plt.title("Test Confusion Matrix")
plt.show()


# ============================================
# 10. Create Confusion Matrix Summary
# ============================================
```

```
confusion_summary = pd.DataFrame({
    "Metric": [
        "True Negative (TN)",
        "False Positive (FP)",
        "False Negative (FN)",
        "True Positive (TP)"
    ],

    "Validation": [
        TN_val,
        FP_val,
        FN_val,
        TP_val
    ],

    "Test": [
        TN_test,
        FP_test,
        FN_test,
        TP_test
    ]
})

print("\n===== CONFUSION MATRIX SUMMARY =====")

display(confusion_summary)
```

```
===== VALIDATION PREDICTIONS =====
```

	Actual Class	Fraud Probability	Predicted Class
0	0	0.215482	0
1	0	0.077170	0
2	0	0.167803	0
3	0	0.093226	0
4	0	0.082598	0
5	0	0.101799	0
6	0	0.160430	0
7	0	0.872360	1
8	0	0.129650	0
9	0	0.246950	0
10	0	0.820061	1
11	0	0.143056	0
12	0	0.104707	0
13	0	0.543520	1
14	0	0.999995	1
15	0	0.189238	0
16	1	0.544689	1
17	0	0.186364	0
18	1	0.262630	0
19	1	0.999651	1

```
===== TEST PREDICTIONS =====
```

	Actual Class	Fraud Probability	Predicted Class
0	0	0.165309	0
1	0	0.138754	0
2	1	0.437629	0
3	0	0.073178	0
4	0	0.129850	0
5	0	0.228718	0
6	0	0.085788	0
7	0	0.179178	0
8	0	0.280091	0
9	0	0.184908	0

9	0	0.184998	0
10	0	0.125168	0
11	0	0.086387	0
12	0	0.226369	0
13	0	0.124474	0
14	0	0.171106	0
15	0	0.123874	0
16	0	0.251432	0
17	0	0.149097	0
18	0	0.103128	0
19	0	0.064369	0

```
===== VALIDATION CONFUSION MATRIX =====
[[168341  14178]
 [  8777   8704]]

TN: 168341
FP: 14178
FN: 8777
TP: 8704

===== TEST CONFUSION MATRIX =====
[[168584  13936]
 [  8831   8649]]

TN: 168584
FP: 13936
FN: 8831
TP: 8649
```